API Assignment Report

Subject: API Plug and Play
Submitted by: Prasiddha Mainali
Date: July 2026

1. Introduction

This assignment is about building a small web application that uses public APIs and an AI model to generate a daily motivational report. The app fetches a random activity and a random joke from two different free APIs, then sends that data to Google Gemini API which writes a short personalized report for the user. The whole thing runs on a simple Python HTTP server with no extra frameworks.

2. Tools and Technologies Used

Component	Technology
Backend	Python (http.server module)
Frontend	HTML, CSS, JavaScript
AI Model	Google Gemini 2.0 Flash
External APIs	Bored API, Official Joke API
Environment	Python 3, no third-party server framework

3. Project Structure

The project has a simple layout:

```
api assignment/
   server.py         - Main Python server (backend)
   .env              - Stores the Gemini API key
   public/
      index.html     - Frontend page
      app.js         - Frontend logic (fetch calls, DOM updates)
      style.css      - Styling
```

4. How It Works

4.1 Backend (server.py)

The backend is a single Python file using the built-in http.server module. It does not use Flask or Django. The server listens on port 8080 and handles three main routes:

- GET /api/config - Checks if a Gemini API key is set in the .env file. Returns a JSON object with a hasKey boolean so the frontend knows the status.
- POST /api/generate - The main endpoint. Accepts activity and joke data in the request body, builds a prompt, calls the Gemini API, and returns the AI-generated report as JSON.
- Static file serving - Any other GET request serves files from the public/ folder (index.html, app.js, style.css).

The server also reads a .env file at startup to load the GEMINI_API_KEY into environment variables. If the key is not in .env, the user can also pass it via a custom header (X-Gemini-Key) from the browser.

4.2 Frontend (public/ folder)

The frontend is plain HTML, CSS, and JavaScript with no frameworks like React or Vue. When the user clicks "Generate My Daily Dose", the JavaScript in app.js does the following:

1. Calls the Bored API (bored-api.appbrewery.com) to get a random activity suggestion.
2. Calls the Official Joke API (official-joke-api.appspot.com) to get a random joke.
3. Sends both results to our backend at /api/generate via a POST request.
4. Receives the AI-generated markdown report, converts it to HTML, and displays it on the page.

There is also a settings modal where the user can enter their own Gemini API key. This gets stored in the browser localStorage and sent as a header with each request.

4.3 Gemini API Integration

The call_gemini function in server.py makes a direct HTTP POST request to the Gemini API endpoint using Python's urllib library (no requests library needed). It sends a prompt that asks the AI to act as a motivational life-coach and write a structured report with four sections: Today's Activity Challenge, Laugh It Off, Your Action Plan, and Fun Fact. The temperature is set to 0.8 to keep responses creative but not too random.

4.4 CORS Handling

Since the frontend and backend run on the same origin (localhost:8080), CORS is not a big issue in development. But the server still adds Access-Control-Allow-Origin: *

headers to every response and handles OPTIONS preflight requests. This makes it easier if someone wants to host the frontend separately later.

5. API Endpoints Summary

Method	Endpoint	Purpose	Response
GET	/api/config	Check API key status	{"hasKey": true/false}
POST	/api/generate	Generate AI report	{"report": "..."}
GET	/*	Serve static files	HTML/JS/CSS files

6. How to Run

1. Make sure Python 3 is installed on your system.
2. Add your Gemini API key in the .env file: GEMINI_API_KEY=your_key_here
3. Open terminal in the project folder and run: python server.py
4. Open browser and go to http://localhost:8080
5. Click "Generate My Daily Dose" and wait for the report.

7. Conclusion

This project shows how to connect multiple APIs together to build something useful. We used two free public APIs to get random data, sent it to an AI model for processing, and displayed the result in a clean web interface. The backend is kept minimal using only Python's built-in libraries, which shows that you don't always need heavy frameworks for simple API-based projects. The Gemini API integration demonstrates how AI models can be used to generate structured content from simple inputs.